

Guías de ejercicios de Ciencias de datos 2026

Docentes

Nombre	contacto
Dr. Marcelo Lares.	marcelo.lares@unc.edu.ar
Dr. Luis Biedma	lbiedma@unc.edu.ar
Lic. Laura Montes	laura.montes@unc.edu.ar

Horarios

Horarios de clases teóricas: martes y jueves de 14hs a 16hs

Horarios de clases prácticas: martes y jueves de 16hs a 18hs

Horarios de consulta: A coordinar con los docentes.

Condiciones de regularidad

- Cumplir con un mínimo de 70 % de asistencia a las clases prácticas
- Aprobar dos exámenes parciales
Habrà un examen de recuperación.

Si no se cumple con los requisitos de regularidad, será posible rendir la materia en condición de libre, para lo cual el examen tendrá ejercicios adicionales a los del examen para regulares.

No se pedirán las guías resueltas.

Condiciones de promoción

La materia se podrá promocionar si se cumplen los siguientes requisitos:

- Tener aprobadas al día de la última clase todas las materias correlativas según el plan de estudio.
- Cumplir con todos los requisitos de regularidad
- Cumplir con un mínimo de 80 % de asistencia a las clases prácticas
- Aprobar las dos evaluaciones parciales con una nota no menor a 6 (seis), y obteniendo un promedio no menor a 7 (siete)

Al promocionar la materia, no será necesario rendir un examen final, y la nota final será la que resulte de promediar las notas de los exámenes parciales.

Ref. Ordenanza HCD N° 4/11

Clave

En estas guías hay tres niveles de ejercicios:

► **1.**

Es altamente recomendable terminar estos ejercicios durante las clases prácticas.

▷ **2.**

Estos ejercicios son práctica adicional para los parciales. El nivel de complejidad es similar al del primer grupo.

◇ **3.**

Ejercicios de dificultad un poco mas elevada. No se debe perder mucho tiempo con ellos pero son una buena practica para los exámenes finales, o si se terminan los otros con tiempo antes de los parciales. Sin embargo estos ejercicios no se explicarán si otros alumnos no han terminado todos los anteriores.

Guía de ejercicios 9: Árboles de Decisión

- 1. Al ingresar al bar el protagonista de este problema, encuentra que sus cinco amigos con quienes se reúne los viernes al salir de trabajar, ya tienen sus bebidas sobre la mesa. María que tiene un cargo de gerente paga la ronda la mitad de las veces. Pablo paga la ronda la cuarta parte de las veces, mientras que Sara y Carlos, que son becarios, pagan indistintamente entre ambos la octava parte de las veces. Juan nunca sacó la billetera desde que se reúnen los viernes.

- (a) ¿Qué fracción de las veces el protagonista paga la ronda?
- (b) Calcular la entropía de la distribución de probabilidad con la que cada uno paga la ronda. ¿Cuál es el número medio de preguntas (de respuesta binaria) que necesitan hacerse en promedio para saber quién paga la ronda?
- (c) Poco después de arribar al bar se suman dos antiguos amigos quienes no vivieron en la ciudad el último año. Ellos deciden que la próxima ronda la debe pagar el protagonista y conociendo que cursa esta materia, lo desafían a predecir que bebida tomará cada uno.

Con la información de las tres variables binarias: sexo, si es o no estudiante y si le gusta o no bailar, y recordando la elección de bebidas de la noche anterior, puede construirse la siguiente tabla:

Bebida	Sexo	Estudiante	Baile
cerveza	M	T	T
cerveza	M	F	T
vodka	M	F	F
vodka	M	F	F
vodka	F	T	T
vodka	F	F	F
vodka	F	T	T
vodka	F	T	T

Usando entropía de información entrenar un árbol de decisión *con lápiz y papel* a partir de la tabla anterior. Registrar todos los valores calculados para elegir las variables en cada nodo del árbol.

- (d) Proponer una poda posible del árbol y calcular la nueva *accuracy* resultante.

- 2. Estudiar la implementación de Árboles de Decisión para clasificación provistas por scikit-learn.
 - (a) ¿Qué algoritmo de árboles de decisión está implementado en scikit-learn? ¿Qué nombre tiene el modelo? ¿Cuál es su principal limitación?
 - (b) Identificar cuál opción del parámetro *criterion* se corresponde con la entropía de Shannon y cuál con Gini como medidas de impureza. ¿Qué parámetros controlan el tamaño del árbol y el *prunning*?

- 3. Entrenar un árbol de decisión usando el iris dataset (usar todos los datos y variables). Emplear entropía de Shannon y luego Gini.
 - (a) Graficar los arboles resultantes usando la información disponible en la entrada de scikit-learn.
 - (b) Identificar la variable utilizada como raíz y el valor de corte resultante en cada caso. Interpretar a partir del conocimiento adquirido sobre este dataset.
 - (c) Utilizando la información disponible en Plot the decision surface of decision trees trained on the iris dataset, estudiar las fronteras de decisión bidimensionales generadas por el árbol de decisión aplicado sólo sobre las variables de sépalo del iris dataset.

- 4. Para evitar que un árbol se ajuste en exceso (overfitting), `DecisionTreeClassifier` proporciona los parámetros `min_samples_leaf` y `max_depth`. La poda usando una función de costo de complejidad proporciona otra opción para controlar el tamaño de un árbol. Esta técnica de poda está regulada por el parámetro `ccp_alpha`. Los valores más altos de `ccp_alpha` aumentan el número de nodos podados.

Con la información provista en Post pruning decision trees with cost complexity pruning, usar el Breast cancer Wisconsin dataset provisto por scikit-learn para:

- (a) graficar la impureza total de las hojas vs. `ccp_alpha`.
- (b) graficar el número de nodos y profundidad del árbol vs. `ccp_alpha`.
- (c) graficar la *accuracy* de clasificación sobre los conjuntos de training y testing vs. `ccp_alpha`.

Guía de ejercicios 10: Métodos de Ensamble

- 1. Para la realización de este práctico resulta útil consultar la ipynb de Sebastian Raschka, pero se propone cambiar de dataset y usar en su lugar el *Breast Cancer Wisconsin* dataset.

Con el fin de evaluar por separado los modelos que integran el ensemble del test del ensemble, separar el 25 % de los datos para test y a su vez dividir el conjunto de entrenamiento, separando de este un 25 % para validación de cada modelo.

- 2. Votación de la mayoría

- (a) Construir tres árboles de decisión con diferentes profundidades usando el parámetro `max_depth = 1, 2, 3`. Luego construir un emsemle con igual peso para esos árboles usando `EnsembleVoteClassifier` de la librería `mlxtend.classifier`; o bien, `VotingClassifier` de `scikit-learn`.
- (b) Reportar la *accuracy* en la validación de cada modelo por separado y en el test del ensemble.

- 3. Bagging: Bootstrap Aggregating

Usando `BaggingClassifier()`, construir una *bolsa* de 500 árboles de decisión, usando todas las características, *bootstrap* con reemplazo y *out-of-bag samples* para estimar el error de generalización. Reportar *accuracy out-of-bag* (OOB) y sobre el set de test.

- 4. Adaptive Boosting (Adaboost)

Utilizar 500 *tree stumps* (`max_depth=1`) para implementar `AdaBoostClassifier()` con el algoritmo SAMME.R, usando todas las variables. Reportar *accuracy* sobre el set de test.

- 5. Gradient Boosting

Implementar `GradientBoostingClassifier()` con los valores default de los parámetros y estudiar el impacto de los parámetros `n_estimators`, `learning_rate`, `max_depth` en *accuracy*. ¿En qué casos se recomienda usar `HistGradientBoostingClassifier()`?

- 6. Random Forests

Implementar `RandomForestClassifier()` y estudiar el significado de los parámetros `max_depth`, `max_features`, `min_samples_leaf` y `min_samples_split`.

Guía de ejercicios 11: Clustering

Scikit-learn provee varios algoritmos de clustering en el módulo `sklearn.cluster`. La propuesta de este práctico es estudiar las implementaciones de K-means, para separar ejemplos no etiquetados de k -grupos disjuntos de varianzas similares y AgglomerativeClustering como ejemplo de clustering jerárquico.

► 1. K-means

- (a) Usar una visualización interactiva online en 2D para ganar intuición sobre el algoritmo de Lloyd.
- (b) Estudiar los parámetros de la función `KMeans()`; en particular, cuál permite elegir los algoritmos: `lloyd`, `elkan`, las opciones de inicialización para los centroides y las opciones de detención. ¿Qué definición de distancia tiene implementada? Identificar la cantidad que se minimiza.
- (c) Para la evaluación de performance de la tarea de clustering se dispone de varias métricas. Estudiar las definiciones de los siguientes scores: `Rand index`, `mutual information`, `homogeneity`, `completeness`, `V measure`, `silhouette coefficient`.

► 2. Utilizando la información disponible en el ejemplo de scikit-learn, aplicar K-means sobre el digits dataset.

- (a) Utilizar como inicialización de los centroides: `k-means++`, `random` y las 10 componentes de PCA sobre el dataset.
- (b) Dado que se dispone de *ground truth* (la clase), implementar las métricas de evaluación estudiadas y reproducir la tabla del ejemplo. Discutir los resultados.
- (c) Proyectar sobre el plano definido por las 2 primeras componentes de PCA para visualizar los datos y el resultado de clusterizar los datos proyectados con k-means.

► 3. Utilizar el coeficiente *silhouette* para seleccionar el número de clusters adecuado, usando como ejemplo datos generados con la función `make_blobs()`.

► 4. Clustering aglomerativo

- (a) Estudiar las nociones de enlace: `ward`, `complete average` y `single`, y las posibles métricas de distancia disponibles para cada opción de enlace en la función `AgglomerativeClustering()`
 - (b) Visualizar el efecto de los distintos *linkages* sobre los datos, usando un embedding 2D de los clusters resultante sobre el `digits dataset`, usando la función provista en el ejemplo de `scikit-learn`.
- 5. **Clustering jerárquico** Aplicar `AgglomerativeClustering()` sobre el `iris dataset` y visualizar el dendograma usando el método disponible en `scipy`.